

AERIS CHECKPOINT 6 — Aero Architecture Bridge Prompt
Use this prompt as the starting context for the next chat.

CONTEXT — WHAT AERIS IS

We are building AERIS: a modular AI-driven aircraft design platform. The target is not a collection of scripts. It is a clean, scalable software architecture for:

- parametric aircraft geometry generation
- modular sampling
- structured dataset generation
- future modular aero analysis
- future ML-ready data products
- future certification-aware and traceable workflows

The project is being built with a strict architectural philosophy:

- modularity first
- determinism first
- CLI-first execution
- structured artifacts / metadata / manifests
- no hidden coupling
- no script spaghetti
- future support for multiple geometry families, samplers, solvers, exporters, and dataset builders

This software direction is already documented in the architecture material: AERIS is intended to act like a modular factory, where interfaces define component classes, implementations plug into registries, and pipelines orchestrate execution without depending on generator-specific code. The goal is extensibility without rewrites and deterministic, traceable outputs suitable for later ML and certification-oriented workflows.

WHY MODULARITY IS NON-NEGOTIABLE

Legacy reference scripts from the older workflow show the real long-term needs of the platform:

- CST-based geometry and section extraction workflows
- multiple downstream geometry reconstructions/exporters (AeroSandbox / OpenVSP / RCAIDE)
- AVL-based aero sweeps with raw outputs and strip data
- scalar feature extraction
- SDF / graph / mesh dataset building
- later GNN consumption

These scripts prove that the future system must support changing:

- geometry generator versions
- extraction logic versions
- export/reconstruction adapters
- sampling methods
- aero solver backends
- dataset builder stages

The old scripts are useful as migration references, but they should NOT be copied into AERIS unchanged because they mix orchestration, core logic, adapters, file semantics, and phase-specific pipeline behavior in monolithic scripts. AERIS is replacing that model with explicit interfaces and registries.

CURRENT STATUS — WHAT IS ALREADY BUILT AND VERIFIED

1. CLI backbone — STABLE

The package has a Typer CLI with separate command groups for geometry, dataset, pipeline, and version reporting. Execution goes through CLI commands, not through scattered production scripts. This matches the intended “single control surface” architecture.

2. Geometry architecture — STABLE

A formal geometry generator base class exists. Generator resolution is done through a registry. The active generator is:

- bwb_segmented_v1



Generator config is resolved through generator-specific typed config construction and validation, not through raw YAML use deep in the pipeline.

The current geometry generator contract includes:

- build_config(raw_config)
- sample_one(config, seed)
- run_full_case(sample, config, output_dir)
- summarize_case(case)

This is a major architectural improvement over earlier duck-typed or duplicated paths. The pipeline now depends on a formal generator contract instead of guessing capabilities via hasattr checks.

3. Deterministic geometry realization — VERIFIED

This is critical:

- sampler seed controls sample generation
- geometry realization is deterministic from explicit sample
- no downstream hidden RNG is used for geometry realization

The current BWB planform implementation explicitly documents and enforces deterministic segment and sweep realization once a sampled design vector exists. This was a deliberate architectural correction from older RNG-contaminated behavior. :contentReference[oaicite:4]{index=4}

4. Modular sampling architecture — STABLE

Dataset sampling has been separated from geometry generation.

A dataset sampler base class and registry exist.

Resolved sampler path:

- CLI/config -> sampler resolver -> sampler registry -> sampler implementation

Validated samplers:

- lhs_v1
- random_v1

Important architectural cleanup already done:

- legacy lhs_seed path was removed from the production spine
 - the real seed path is now sampler_seed
 - seeding belongs to the sampler interface, not specifically to LHS
-

This means the pipeline is now sampler-modular, not just “LHS with extra decorations.”

5. Dataset generation pipeline — STABLE

The dataset pipeline is now a real orchestrator:

raw config -> generator resolution -> typed config -> sampler resolution -> explicit design samples -> generator.run_full_case() -> artifacts + metadata + failures + manifest

This is important:

- dataset generation does NOT reimplement geometry logic
- generator.run_full_case() remains the single geometry realization path
- metadata and failure tracking are structured and explicit
- resolved config snapshots are written
- dataset manifests are written
- paths and logs are standardized

The dataset pipeline survived architecture hardening and remains functional.

6. Validation and hardening checks — PASSED

The following checks were run and passed:

- bad config failure handling
- missing config CLI rejection
- invalid sampler failure handling
- sampler modularity check (lhs_v1 vs random_v1)
- geometry-only stress test at 500 cases with plotting disabled
- AeroSandbox-enabled batch validation path

The geometry-only no-plot stress test produced:

- requested_n = 500
 - attempted_n = 500
 - succeeded_n = 500
 - failed_n = 0
 - no missing files
 - no duplicate IDs
 - metadata rows correct
-

- failure rows correct

This confirms that the geometry/export path is stable under moderate batch load.
:contentReference[oaicite:7]{index=7}

7. Plotting is now recognized as optional hot-path overhead
Stress testing revealed that plotting was the main runtime tax.
The no-plot dataset path is the correct default for serious batch generation.
Visualization is not part of the core production hot path and must remain optional.

8. AeroSandbox geometry build path — VERIFIED

A smaller validation run with AeroSandbox-enabled generation confirmed that:

- aspect_ratio_aerosandbox is populated
- n_xsecs_aerosandbox is populated
- geometry + AeroSandbox build path succeeds cleanly in batch mode
- plot_path can remain null when plots are disabled, which is correct

This means AERIS currently has:

- a pure geometry/export path
- a geometry + AeroSandbox object build path

Both are working.

9. Smoke pipeline meaning was corrected conceptually

It was discovered that the old smoke pipeline only loaded YAML, copied config, and wrote a manifest. That was too shallow and produced false confidence.

The smoke concept should mean:

raw config -> generator resolution -> typed validation -> one sample -> one geometry case -> artifact path -> success/failure manifest

This matters because smoke testing must exercise the real minimum viable path, not just file I/O.

CURRENT ARCHITECTURAL TRUTH

The repo now has a clean enough foundation to move forward.
The following are now true:

- geometry generators are plugin-based
- dataset samplers are plugin-based
- dataset generation reuses the real generator path
- determinism is enforced correctly
- dataset metadata / failures / manifests are structured
- no-plot batch generation is fast and stable
- AeroSandbox object construction path works
- the architecture is no longer pretending to be modular — it actually is modular at the geometry/sampling layer

WHAT IS STILL LIMITED / NOT YET FULLY GENERAL

Be honest and explicit:

1. Only one real geometry family is implemented in AERIS right now

Current active family:

- bwb_segmented_v1

2. Sampler plugins are architecturally modular, but current sampler implementations are still tied to the BWB config/sample types

That is acceptable for now, but it is not yet full geometry-family-agnostic sampling.

3. Export/reconstruction adapters are not yet formal plugin families

AeroSandbox object construction exists, but there is not yet a formal geometry-view/export adapter registry.

4. Dataset builders are not yet formal plugin families

The current dataset pipeline writes geometry metadata artifacts.

Future SDF / graph / mesh / aero-label builders are not yet formalized inside

AERIS.

5. Aero solver architecture is only a shell

The repo already has an aero folder scaffold, but the real architecture and first solver path still need to be designed properly. :contentReference[oaicite:9]{index=9}

KEY LESSONS FROM LEGACY REFERENCE SCRIPTS

The older prototype scripts prove that AERIS must eventually support more plugin families than just generators and samplers.

These future plugin/seam families are justified by the legacy scripts:

A. Geometry generators

Examples:

- current segmented BWB
- future CST surface-based geometry

The CST extraction script shows a fundamentally different geometry philosophy with exact section extraction and transform-aware geometry equivalence validation. That should be a separate geometry family/version in AERIS, not crammed into the current one.

B. Geometry views / export adapters

Examples:

- AeroSandbox view
- OpenVSP CSV / airfoil export view
- RCAIDE view

The old CST-to-ASB/VSP and CST-to-RCAIDE scripts prove that the same geometric intent may need multiple downstream representations. These should become adapters/views, not generator-internal hacks.

C. Aero solver plugins

Examples:

- aerosandbox_avl
-

- later VSPAERO / CFD adapters

The old AVL workflow proves the solver layer needs structured adapters with raw file capture, panel settings, strip parsing, totals, and derived outputs. This must be a first-class plugin family.

D. Dataset builders

Examples:

- geometry metadata builder
- aero label builder
- SDF builder
- graph/mesh builder

The old full-dataset builder proves that future data products will not be one flat metadata CSV. They will be multiple builder families with their own semantics.

WHAT SHOULD BE PREPARED NOW VS LATER

Prepare NOW:

1. Aero solver plugin architecture
2. Lightweight geometry-view/export seam
3. Keep future dataset-builder modularity in the design

Do NOT fully implement yet:

1. Extraction-policy registry
2. Reverse-fitting / inverse geometry plugins
3. ML model/training plugin systems

The correct balance is:

- prepare the next real extension seams
- do not over-abstract into an empty plugin cathedral

NEXT PHASE — CHECKPOINT 6

The next phase is:

CHECKPOINT 6 — Aero Architecture Design

This must be done as a system architect, not as a script hacker.

The next chat must design:

1. Aero input/output contracts

Need explicit models for:

- geometry input representation
- flight condition / operating point
- solver settings
- aero result payload
- failure payload
- provenance / metadata payload

2. Aero solver base class

Need a formal solver contract similar in spirit to geometry/sampler plugins.

3. Aero solver registry

Need:

- register solver by ID
- get solver by ID
- list available solvers

4. First solver implementation

First solver should be:

- aerosandbox_avl

Not CFD, not pressure fields, not ML.

The first goal is a robust low-order aero path.

5. Failure handling

Need clean states such as:

- success
 - solver_failed
-

- invalid_output

Need explicit capture of:

- raw stdout / stderr if applicable
- solver metadata
- runtime
- convergence / validity status

6. Dataset integration

The future pipeline should become:

sample -> geometry -> optional aero -> metadata/artifacts

This should not fork into a second ugly pipeline if avoidable.

7. Geometry-view/export seam

The aero solver should NOT force geometry internals to be rewritten.

A lightweight view/adaptor seam should be considered so future OpenVSP / RCAIDE / other reconstructions fit naturally.

CRITICAL DESIGN QUESTIONS FOR NEXT CHAT

The next chat must answer these cleanly:

1. What is the canonical geometry input to an aero solver?

Candidates:

- AeroSandbox Airplane object
- a typed AERIS geometry view
- section-based representation

Decision must be made intentionally.

2. What are the minimum standardized aero outputs for v1?

Suggested first outputs:

- CL
 - CD
-

- C_m
- L_{over_D}
- validity/convergence/status metadata

3. What aero outputs are future, but not v1?

Examples:

- strip forces
- stability derivatives
- pressure distributions
- C_p fields
- richer spanwise data

4. How will solver-specific raw outputs be stored without polluting the common interface?

5. How will solver provenance/version/settings be tracked for ML and traceability?

6. How should the future geometry-view/export seam relate to the aero layer?

INSTRUCTION FOR THE NEXT CHAT

Continue from this exact status.

Do not restart from generic advice.

Do not propose random refactors detached from current architecture.

Do not collapse back into script-style orchestration.

Do not over-design speculative plugin families that are not yet needed.

Act as a system architect working on the next real extension seam.

Use the current validated state as the base:

- geometry generator plugin architecture works
 - sampler plugin architecture works
 - dataset orchestration works
-

- no-plot stress path works
- AeroSandbox geometry build path works

Now design the aero architecture cleanly, with enough foresight to support future exporters/views and later dataset builders, but without premature abstraction bloat.
